MASTER ARCHITECTURE AND CONSTITUTION

AGBOFA

The governed digital product operating system. Intelligence is not authority.

Document issue v5.1

Operative issue -- implement against this

Day 0 -- pre-implementation freeze

Human Owner + Senior System Architect

> *AGBOFA is not a Claude wrapper. Claude is replaceable intelligence. AGBOFA owns authority, governance, execution, verification, evidence, audit, memory, and recovery.*

00 AGBOFA Master Architecture & Constitution

The governed digital product operating system.

Document issue: v5.1 -- Operative -- implement against this

Received freeze: v5.0 -- Preserved, not repealed

First increment: Day 60 -- Maturity M0--M2 certified

Fundamental distinction. The master architecture describes AGBOFA from foundation through enterprise maturity. The 60-day plan describes the first implementation increment, not the limits of AGBOFA.

> *AGBOFA is not a Claude wrapper. Claude is replaceable intelligence. AGBOFA owns authority, governance, execution, verification, evidence, audit, memory, and recovery.*

v5.1 does not replace the owner's vision and does not repeal v5.0. It is a minor issue of the same constitution: architecture versus increment is stated as law, and numbering is disambiguated so an implementer cannot confuse document issue, maturity, autonomy, evidence, or the 60-day schedule.

01 Versioning law

Five numbering systems. They are not interchangeable.

Binding issue. An implementer, bid, or agent is bound by document issue v5.1 unless the Human Owner explicitly re-freezes v5.0. v5.1 is a MINOR bump: no article is repealed. A MAJOR (v6.0) is required to change, add, or weaken an article.

Five independent numbering systems

System | Prefix | Range | Means | Must not be read as

- System: Document issue
- Prefix: v
- Range: vMAJOR.MINOR.PATCH
- Means: Which freeze of this Constitution is in force
- Must not be read as: Product version, maturity, or Day count

- System: Enterprise maturity
- Prefix: M
- Range: M0--M5
- Means: How much of the target architecture is certified operational
- Must not be read as: Agent autonomy or document issue

- System: Agent autonomy
- Prefix: A
- Range: A0--A5
- Means: What a given agent is allowed to do
- Must not be read as: Platform maturity

- System: Evidence
- Prefix: E
- Range: E0--E4
- Means: Strength of proof required for a risk class
- Must not be read as: Autonomy or maturity

- System: Delivery

- Prefix: Phase / Block / Gate / Day
- Range: 1--4 / 1--10 / 1--10 / 1--60
- Means: First increment schedule
- Must not be read as: Document issue v1--v60

Issues in this reader

Issue | Status | What it is

v5.0 | Received freeze | Owner + architect constitutional formalization, preserved as submitted

v5.1 | Operative | Same constitution. Architecture vs increment stated as law. Numbering disambiguated. Layers, agents, factory states, and security constitution specified

Mapping from v5.0 notation

v5.0 wrote | v5.1 writes | System

Autonomy L0--L5 | A0--A5 | Agent autonomy

Maturity L0--L5 | M0--M5 | Enterprise maturity

"Version 1 / Version 2" as reader labels | v5.0 received / v5.1 operative | Document issue -- never reuse v1/v2 for the reader

When the number increments

Bump | Required when | Example

Major (v6.0) | An article is added, repealed, or substantively changed | New Article 17, or weakening Article 3

Minor (v5.1) | Clarification, new specified capability, or numbering disambiguation without changing articles | This issue

Patch (v5.1.1) | Errata that do not change meaning | Typo in a schema name

- v5.1 is not "the second version of the AGBOFA product."
- M2 is not A2. A factory at maturity M2 may still run agents at autonomy A1.
- Gate 10 is not maturity M5. Gate 10 certifies the first increment.
- Day 60 is not document version 60.
- Phase 1 is the first delivery increment, not constitution v1.0.
- Historical v1.0--v4.0 are superseded lineage, not toggleable reader editions.
- A product passport records constitutionVersion. Work against a product is bound to that freeze.

02 How to read this document

Architecture is permanent. The roadmap is an increment.

> Read in this order. Parts A--E are the system. Part F is the first increment against that system. Part G is how the system itself may change. Do not implement Part F as if it were the architecture.

What is permanent

- The 16 articles and 15 principles
- Intelligence != authority
- Provider independence (Claude Business is the initial adapter, not the identity of AGBOFA)
- The operating stack from Human through Learning
- Enterprise maturity M0--M5 as the target architecture

What is the first increment

- Single controlled deployment environment
- Claude Business as the first Intelligence Provider behind the contract
- A limited supported technology profile (web app, API service, database templates)
- Twelve initial specialist roles at A0/A1 (Verification Agent at A2)
- Gates 1--10 proving the constitutional core and the product factory (maturity M0--M2)

What is in architecture but not in Day 60 operations

- Multi-region infrastructure
- Live multi-provider routing
- Enterprise SDK, federation, marketplace
- Hundreds of agents
- Full SOC2/GDPR operational certification

Those items remain in Part E. Phase 1 must leave the seams (contracts, adapters, registries, isolation, audit) so they can be switched on without rewriting the Constitution.

03 What v5.1 changes -- and what it does not

Minor issue of the received freeze. Articles unchanged.

> v5.0 is not thrown away. The received freeze is the conceptual foundation. v5.1 is a constitutional expansion and formalization -- a MINOR issue -- not a smaller MVP document and not a new product version.

Change in v5.1 | Why

Versioning law (this front matter) | Stops v5.0/v5.1, maturity, autonomy, evidence, and Day 60 from being read as one scale

Rename maturity L->M and autonomy L->A | v5.0 used L0--L5 for both; they are different systems

Promote the architecture-versus-increment sentence to law | Stops implementers from reading the 60-day list as the edge of AGBOFA

Rewrite F.1 Day-60 objective | Received text asked for 100% of every enterprise function in 60 days -- that reintroduces the v4 ambiguity

Specify every operating layer | Purpose, I/O, permissions, prohibitions, failures, security, audit

Agent specification framework | Roles become contracts, not a list of names

Product factory state machine | Lifecycle becomes implementable states, not a poster

Security constitution as its own section | Threats were implied; they are now named

Passport as a formal contract, including constitutionVersion | Identity of an AGBOFA product is executable and bound to a freeze

Unchanged on purpose: articles 1--16, principles P1--P15, risk classes, evidence levels E0--E4, ten gates, red-team catalog, schemas, repository layout, and the owner’s vision of a governed digital product operating system. Autonomy ranks and maturity ranks keep their meanings; only the prefix changes.

A.1 Preamble

AGBOFA is a governed digital product operating system. This Constitution is supreme.

AGBOFA (Autonomous Governed Blockchain-Oriented Factory Architecture) is established as a governed digital product operating system. This Constitution is the supreme authority governing all actors, systems, agents, models, and processes within the AGBOFA ecosystem.

The purpose of this Constitution is to ensure that artificial intelligence may autonomously design, build, and operate digital products safely, verifiably, and constitutionally, while human sovereignty remains absolute and every consequential action carries evidence, accountability, and the possibility of audit.

> Supremacy. No agent, model, subsystem, or process may violate this Constitution. No authority may be exercised except through the mechanisms defined herein.

A.2 Permanent architectural principles

P1--P15 do not change with maturity, timeline, or provider.

These principles are permanent. They do not change with maturity level, implementation timeline, or provider selection.

ID | Principle | Statement

P1 | Human Sovereignty | Humans are ultimately accountable for AGBOFA’s existence and major consequential actions.

P2 | Constitutional Supremacy | The Constitution governs all actors, including AGBOFA itself. No agent, model, or process may violate it.

P3 | Intelligence Is Not Authority | No AI provider, model, agent, or generated output automatically possesses authority. Authority must be explicitly granted through constitutional mechanisms.

P4 | Authority Is Explicit and Traceable | Every consequential action requires an identifiable authority source that can be traced through the governance chain.

P5 | Capability Is Scoped and Constrained | An actor can only perform actions for which it has been granted both authority and technical capability, within defined boundaries.

P6 | Execution Is Isolated | Intelligence never receives unrestricted execution access. All execution occurs through governed, isolated channels.

P7 | Verification Is Independent | Important outcomes must be independently verified before they are accepted as true or successful.

P8 | Evidence Precedes Trust | Claims about system state, product quality, or operational success require evidence. Trust is earned, not assumed.

P9 | Audit Is Persistent and Tamper-Evident | Consequential actions create durable audit records that cannot be silently modified or deleted.

P10 | Governance Cannot Be Weakened by the Governed | No agent, model, or subsystem may modify the rules that govern its own authority.

P11 | Autonomy Is Earned | Autonomy increases only according to demonstrated evidence of reliability, safety, and constitutional compliance.

P12 | Learning Requires Verified Outcomes | AGBOFA cannot convert an AI assumption or recommendation into institutional knowledge merely because an AI generated it.

P13 | Tenant Isolation Is Absolute | Products, tenants, organizations, and environments are isolated by default. Cross-boundary access requires explicit, governed authorization.

P14 | Recovery Is Constitutional | The system must be capable of freezing, recovering, rolling back, and restoring critical state. Recovery procedures are themselves governed.

P15 | Provider Independence | AGBOFA’s architecture never equates its intelligence with any specific AI provider. Providers are replaceable through defined contracts.

A.3 Constitutional articles

Articles 1--16. The operative law of every actor, agent, and subsystem.

The sixteen articles are the operative law of AGBOFA. They bind every actor. They do not change with maturity, timeline, or provider.

Article 1 Human Sovereignty

Humans remain ultimately accountable for AGBOFA's existence, operation, and major consequential actions.

Scope. All AGBOFA operations, decisions, and actions.

Prohibitions

- AGBOFA shall not take actions that remove or diminish human accountability.
- No agent may claim to act without human oversight for constitutionally significant actions.

Requirements

- Human approval is required for all actions classified as HIGH or CRITICAL risk.
- A designated human owner must possess emergency freeze authority at all times.
- Human decisions that override agent recommendations must be recorded with rationale.

Enforcement. The Trust Kernel shall reject any action classified as HIGH or CRITICAL risk that lacks valid human approval.

Violation consequence. System freeze and constitutional review.

Article 2 AGBOFA Sovereignty

AGBOFA controls the ecosystem's authority and governance mechanisms.

Scope. All authority grants, governance rules, policy definitions, and capability assignments.

Prohibitions

- No external system may directly grant authority within AGBOFA.
- No agent may self-grant authority.

Requirements

- All authority flows through the AGBOFA Governance Engine.
- All governance rules are versioned and stored in the Constitution repository.
- All authority grants are recorded as audit events.

Enforcement. The Authority Kernel shall reject any authority grant that does not originate from an authorized governance source.

Violation consequence. Invalid authority revocation and security investigation.

Article 3 Intelligence Is Not Authority

No AI provider, model, agent, recommendation, inference, or generated output automatically receives authority.

Scope. All AI-generated outputs, recommendations, plans, and proposals.

Prohibitions

- AI outputs shall not be treated as commands.
- AI recommendations shall not be auto-executed without governance evaluation.
- AI confidence shall not substitute for authority.

Requirements

- All AI outputs enter the system as PROPOSALS.
- Proposals must pass through governance evaluation before execution.
- The distinction between intelligence and authority must be explicit in every agent interaction.

Enforcement. The Execution Broker shall reject any action whose authority chain does not trace to a valid, non-AI authority source.

Violation consequence. Action rejection and agent capability suspension.

Article 4 Authority Is Explicit

Every consequential action requires an identifiable authority source.

Scope. All consequential actions within AGBOFA.

Prohibitions

- Implicit authority is prohibited.
- "The agent knew what to do" is not an authority source.
- Default allow is prohibited.

Requirements

- Every action carries an authority chain.
- Authority chains must be traceable to a human or constitutional source.
- Authority grants must have defined scope, duration, and revocation conditions.

Enforcement. Actions lacking valid authority chains are rejected at the Trust Kernel.

Violation consequence. Action rejection and audit flag.

Article 5 Capability Is Scoped

An actor can only perform actions for which it possesses the required capability.

Scope. All actor actions within AGBOFA.

Prohibitions

- Broad capabilities (e.g., "agent has access to everything") are prohibited.
- Capability escalation without governance approval is prohibited.
- Capability reuse across product boundaries is prohibited by default.

Requirements

- All capabilities are defined with explicit scope, environment, and expiry.
- Capability tokens are required for all execution actions.
- Capability tokens are short-lived and renewable only through governance.

Enforcement. The Capability Manager shall reject any execution attempt with missing, expired, or out-of-scope capability tokens.

Violation consequence. Capability revocation and security audit.

Article 6 Execution Is Isolated

Intelligence does not receive unrestricted execution access.

Scope. All execution activities.

Prohibitions

- Direct agent access to production systems is prohibited.
- Execution outside isolated workspaces is prohibited.
- Bypassing the Execution Broker is prohibited.

Requirements

- All execution occurs through the Execution Broker.
- All execution occurs in isolated workspaces.
- All execution carries resource limits, timeouts, and failure handling.

Enforcement. The Execution Broker shall reject any direct execution attempt that bypasses its governance chain.

Violation consequence. Execution rejection and agent quarantine.

Article 7 Execution Is Not Success

An executed action does not constitute proof of correctness.

Scope. All execution outcomes.

Prohibitions

- Treating "the command ran" as "the command was correct."
- Accepting agent claims of success without independent verification.
- Skipping verification for consequential actions.

Requirements

- All consequential actions require independent verification.
- Verification must be performed by a component separate from the executing agent.
- Verification results must be recorded as evidence.

Enforcement. The Verification Engine shall flag any consequential action lacking verification evidence.

Violation consequence. Action marked unverified and blocked from promotion.

Article 8 Verification Is Independent

Important outcomes must be independently verified.

Scope. All important outcomes, including builds, tests, deployments, and data operations.

Prohibitions

- Agents verifying their own work.
- Verification by the same component that performed the action.
- Verification that only checks surface-level success.

Requirements

- Verification is performed by dedicated verification agents or systems.
- Verification checks both functional correctness and constitutional compliance.
- Verification produces evidence that is stored and auditable.

Enforcement. Outcomes lacking independent verification are blocked from advancing to the next lifecycle stage.

Violation consequence. Outcome rejection and re-execution requirement.

Article 9 Evidence Precedes Trust

Claims about system state require evidence.

Scope. All claims about system state, product quality, or operational success.

Prohibitions

- Accepting agent assertions without evidence.
- Treating absence of errors as evidence of correctness.
- Storing claims without supporting evidence.

Requirements

- Every consequential claim carries evidence.
- Evidence must be machine-verifiable where possible.
- Evidence must be stored in the Evidence Store with proper provenance.

Enforcement. Claims lacking evidence are marked unverified and cannot be used for decision-making.

Violation consequence. Claim rejection and investigation.

Article 10 Audit Is Persistent

Consequential actions create durable audit records.

Scope. All consequential actions.

Prohibitions

- Audit record modification or deletion.
- Audit bypass for any reason.
- Tampering with audit timestamps, hashes, or chains.

Requirements

- All consequential actions generate audit events.
- Audit events are append-only.
- Audit events are hash-chained for tamper evidence.
- Audit records include: who, which agent, which authority, which capability, which product, which environment, what action, when, why, what happened, what was verified, what evidence exists.

Enforcement. The Audit System shall detect any tampering attempt and trigger a security alert.

Violation consequence. System freeze and security investigation.

Article 11 Governance Cannot Be Weakened by the Governed

An agent cannot modify the rules that govern its own authority.

Scope. All governance rules, policies, and authority definitions.

Prohibitions

- Agent modification of its own authority scope.
- Agent modification of governance rules that apply to itself.
- Agent creation of new capabilities that bypass governance.

Requirements

- Governance changes require human approval.
- Governance changes are versioned and audited.
- Governance changes are subject to constitutional review.

Enforcement. The Governance Engine shall reject any governance change originating from a governed agent.

Violation consequence. Change rejection and agent suspension.

Article 12 Autonomy Is Earned

Autonomy increases only according to defined evidence and policy.

Scope. All agent autonomy levels.

Prohibitions

- Automatic autonomy grants.
- Autonomy increases without evidence of reliability.
- Autonomy based on AI self-assessment.

Requirements

- Autonomy levels are defined in the Constitution.
- Autonomy increases require evidence of successful governed operation.
- Autonomy increases require human approval.

Enforcement. The Trust Kernel shall enforce autonomy levels and reject actions exceeding authorized autonomy.

Violation consequence. Action rejection and autonomy level reduction.

Article 13 Learning Requires Verified Outcomes

AGBOFA cannot convert an AI assumption into institutional knowledge merely because the AI generated it.

Scope. All institutional knowledge and learning.

Prohibitions

- Direct promotion of AI outputs to knowledge without verification.
- Treating AI confidence as knowledge validity.
- Storing unverified claims in institutional memory.

Requirements

- Knowledge candidates must pass through verification.
- Verified outcomes must be linked to their evidence.
- Knowledge must be versioned and auditable.

Enforcement. The Memory System shall reject knowledge candidates lacking verification evidence.

Violation consequence. Knowledge candidate rejection.

Article 14 Tenant Isolation

One product, tenant, organization, or environment cannot improperly access another.

Scope. All cross-tenant, cross-product, and cross-environment interactions.

Prohibitions

- Cross-tenant data access by default.
- Cross-product agent access by default.
- Cross-environment capability use by default.

Requirements

- All tenant data is isolated by default.
- Cross-boundary access requires explicit, governed authorization.
- Tenant isolation is tested continuously.

Enforcement. The Isolation Guard shall detect and block any cross-tenant access attempt lacking explicit authorization.

Violation consequence. Access denial, isolation breach alert, and security investigation.

Article 15 Recovery Is Constitutional

The system must be capable of freezing, recovering, rolling back, and restoring critical state.

Scope. All critical state, including products, databases, configurations, and governance rules.

Prohibitions

- Operating without defined recovery procedures.
- Recovery procedures that bypass governance.
- Recovery actions that are not audited.

Requirements

- Recovery procedures are defined for all critical state.
- Recovery actions are authorized and audited.
- Recovery testing occurs regularly.
- Emergency freeze capability exists and is testable.

Enforcement. The Recovery System shall reject recovery actions lacking proper authorization and audit trail.

Violation consequence. Recovery action rejection and constitutional review.

Article 16 Provider Independence

AGBOFA’s architecture never equates its intelligence with any specific AI provider.

Scope. All intelligence integration points.

Prohibitions

- Hard-coding provider-specific APIs in core AGBOFA components.
- Requiring provider-specific features for constitutional operation.
- Treating any provider as irreplaceable.

Requirements

- All intelligence access flows through the Intelligence Contract.
- Provider adapters are pluggable and versioned.
- Provider switching is possible without core changes.

Enforcement. The Intelligence Contract shall reject any direct provider access that bypasses the abstraction layer.

Violation consequence. Architecture violation flag and refactoring requirement.

A.4 Autonomy levels

Autonomy is earned. No agent starts above proposal. L in the freeze is autonomy, not maturity.

Agent autonomy is the A scale. It is a per-actor grant, earned under Article 12. It is not enterprise maturity (M) and not a document-issue number.

> A is not M. A0--A5 are agent autonomy grants. M0--M5 are enterprise maturity of the platform. Mixing them is the numbering error the received freeze produced by using L for both.

Level | Name | Description | Requirements | v5.0 wrote

- Level: A0
- Name: Advisory
- Description: Agent provides recommendations only. No execution authority.
- Requirements: None -- default state for intelligence-only roles.
- v5.0 wrote: L0

- Level: A1
- Name: Proposal
- Description: Agent can submit formal proposals for governance evaluation.
- Requirements: Proven recommendation quality.
- v5.0 wrote: L1

- Level: A2
- Name: Governed Execution
- Description: Agent can execute within pre-approved scope with governance checks.
- Requirements: Evidence of safe A1 operation.
- v5.0 wrote: L2

- Level: A3
- Name: Autonomous Within Scope
- Description: Agent can execute without per-action approval within defined scope.
- Requirements: Evidence of safe A2 operation + human approval.
- v5.0 wrote: L3

- Level: A4

- Name: Autonomous with Verification
- Description: Agent can execute and self-verify within scope, with periodic external verification.
- Requirements: Evidence of safe A3 operation + human approval.
- v5.0 wrote: L4

- Level: A5
- Name: Full Governed Autonomy
- Description: Agent can operate autonomously within constitutional limits.
- Requirements: Evidence of safe A4 operation + human approval + constitutional review.
- v5.0 wrote: L5

Grant ceiling. No agent starts above A1. Named agents in the operative catalogue are A0 (advisory intelligence), A1 (proposal), or A2 (independent verification within pre-approved scope). A3--A5 are defined, not granted at Day-60 certification.

A.5 Risk classifications

CRITICAL, HIGH, MEDIUM, LOW. Approval is a function of class, not of model confidence.

Every consequential action is classified before governance evaluation. Classification is a Trust Kernel function. AI confidence is not a risk class.

Classification | Description | Approval required

CRITICAL | Actions affecting constitutional integrity, production data, security boundaries, or system-wide state. | Human + Constitutional Review

HIGH | Actions affecting product deployments, schema changes, credential operations, or cross-tenant access. | Human

MEDIUM | Actions affecting code changes, test execution, or non-production environments. | Governance Engine

LOW | Actions affecting documentation, analysis, or read-only operations. | Capability Token only

Default-deny. Unclassified consequential actions are treated as HIGH until classified. Default-allow is prohibited (Article 4).

B.1 System overview

Human -> Constitution -> Governance -> Authority -> Capabilities -> Intelligence -> Planning -> Execution -> Verification -> Evidence -> Audit -> Memory -> Learning.

Authority flows downward. Evidence and audit flow back up. No layer may skip the layers above it. Intelligence sits below capability; it never sits above authority.

1. Human -- Sovereign Authority
2. Constitution -- Supreme Governing Law
3. Governance -- Policy & Rule Engine
4. Authority -- Explicit Grant System
5. Capabilities -- Scoped Token System
6. Intelligence -- AI Provider Abstraction
7. Planning -- Governed Action Plans
8. Execution -- Isolated Broker / Workspace
9. Verification -- Independent Validation
10. Evidence -- Immutable Proof Store
11. Audit -- Append-Only Trail
12. Memory -- Verified Knowledge
13. Learning -- Institutional Growth

Read direction. A proposal originates in Intelligence or Planning and must climb back through Governance and Authority before Execution will admit it. Learning cannot grant autonomy or rewrite the Constitution.

B.2 Core subsystems

The kernels and engines that enforce the Constitution in software.

Subsystems are the implementable counterparts of the stack. Each maps to one or more articles. Absence of a subsystem is a constitutional defect, not a backlog item.

Subsystem | Purpose | Key components

Trust Kernel | Enforce constitutional rules | Auth, Authorization, Policy Engine, Risk Classification

Authority System | Manage explicit authority grants | Authority Chains, Delegation, Revocation

Capability System | Issue scoped capability tokens | Token Issuance, Scope Validation, Expiry Management

Execution Broker | Control all execution | Action Classification, Governance Check, Workspace Allocation

Verification Engine | Independently verify outcomes | Test Runners, Build Verification, Deployment Verification

Evidence Store | Store immutable evidence | Hash-chained Storage, Evidence Validation

Audit System | Maintain append-only audit trail | Event Logging, Hash Chaining, Tamper Detection

Intelligence Contract | Abstract AI providers | Provider Adapters, Agent Runtime, Context Management

Product Factory | Transform ideas into products | Discovery, Requirements, Blueprint, Architecture, Planning

Memory System | Store verified knowledge | Knowledge Candidates, Validation, Institutional Memory

Recovery System | Enable constitutional recovery | Freeze, Rollback, Restore, Emergency Procedures

Operations System | Manage running products | Monitoring, Incident, Compliance, Lifecycle

B.3 Intelligence architecture

Providers are adapters. Outputs are proposals. Autonomy is A, not L.

All model access flows through the Intelligence Contract. Provider adapters are pluggable. Agent outputs enter the system as proposals, never as commands (Article 3, Article 16).

1. Intelligence provider
2. Provider adapter
3. Intelligence Contract
4. Agent runtime
5. Agent role
6. Agent context
7. Agent task
8. Recommendation / plan / output (PROPOSAL)

Initial agent roles

A, not L. The received freeze wrote L1 / L2 on this roster. Those cells were autonomy, not maturity. The operative issue uses A1 / A2. Advisory intelligence roles (A0) are catalogued in Part C.

Agent | Purpose | Autonomy

Product Discovery Agent | Transform ideas into requirements | A1

Requirements Agent | Define and validate requirements | A1

Domain Analyst Agent | Analyze domain models | A1

Architecture Agent | Propose system architecture | A1

Database Agent | Design and manage data models | A1

UX Agent | Design user experience | A1

Security Agent | Identify and address security concerns | A1

Repository Agent | Manage code repositories | A1

QA Agent | Design and execute tests | A1

Verification Agent | Independently verify outcomes | A2

Deployment Agent | Manage deployments | A1

Operations Agent | Monitor and manage operations | A1

B.4 Product factory lifecycle

Idea to retirement under governance. v5.1 adds Discovery, Requirements, and Architecture state machines.

The factory is a governed pipeline, not a prompt. Human approval sits after architecture and before implementation. Verification and evidence sit after execution and before deployment.

1. Idea
2. Discovery
3. Requirements
4. Domain model
5. Product Passport
6. Blueprint
7. Architecture
8. Human approval
9. Implementation plan
10. Execution
11. Build
12. Test
13. Security
14. Verification
15. Evidence
16. Deployment approval
17. Deployment
18. Post-deployment verification
19. Operations
20. Learning
21. Improvement
22. Retirement

Three factory stages are explicit state machines. Transitions are proposals until governance (and, for architecture, human approval) attaches. There is no implicit ‘the agent knew the next state.’

Discovery

- Idle -> intake
- Intake -> analyze
- Analyze -> propose / reject
- Propose -> govern
- Governance evaluation -> approved / returned
- Returned -> analyze
- Approved -> complete
- Rejected ->
- Complete ->

Requirements

- Idle -> classify
- Classify -> assumptions
- Track assumptions -> propose
- Propose -> govern
- Governance evaluation -> approved / returned
- Returned -> classify
- Approved -> complete
- Complete ->

Architecture

- Idle -> draft
- Draft -> contracts
- Contract schemas -> propose
- Propose -> human
- Human approval -> approved / returned
- Returned -> draft
- Approved -> complete
- Complete ->

B.5 Execution path

Proposal to audit. Nothing runs that has not passed classification, governance, authority, capability, and approval.

Execution is admitted only through the Execution Broker, only inside an isolated workspace, and only against a valid authority chain and in-scope capability token (Articles 4, 5, 6). A result is a claim, not success (Article 7).

1. Agent proposal
2. Action classification
3. Risk assessment
4. Governance evaluation
5. Authority check
6. Capability check
7. Approval check
8. Capability token
9. Execution Broker
10. Isolated workspace
11. Worker
12. Result
13. Verification
14. Evidence
15. Audit

B.6 Verification and evidence

Independent checks. Evidence levels E0--E4. E0 is unacceptable for every consequential claim.

Verification types

Type | Purpose | Performed by

Functional | Verify functional requirements | Verification Agent

Unit | Verify individual components | Automated Test Runner

Integration | Verify component interactions | Automated Test Runner

E2E | Verify user workflows | Playwright Runner

Security | Verify security controls | Security Agent

Performance | Verify performance requirements | Performance Runner

Deployment | Verify deployment success | Deployment Verifier

Data Integrity | Verify data operations | Data Verifier

Recovery | Verify recovery procedures | Recovery Tester

Configuration | Verify configuration correctness | Configuration Verifier

Compliance | Verify constitutional compliance | Compliance Checker

Evidence levels

Evidence numbering is E0--E4 in both the received freeze and the operative issue. It is not renamed. It is not autonomy and not maturity.

Level | Description | Required for

E0 | No evidence -- agent assertion only | Nothing (unacceptable)

E1 | Automated test output | LOW risk actions

E2 | Test output + configuration snapshot | MEDIUM risk actions

E3 | Test output + configuration + security scan | HIGH risk actions

E4 | Full evidence package + independent verification | CRITICAL risk actions

> E0 is never enough. Claims lacking evidence are marked unverified and cannot be used for decision-making (Article 9). Absence of errors is not evidence of correctness.

B.7 Memory and learning

AI saying something does not become AGBOFA knowledge.

1. Raw observation
2. Evidence
3. Verified outcome
4. Knowledge candidate
5. Validation
6. Institutional knowledge
7. Future decision support

> Verified outcomes only. AI saying something does not become AGBOFA knowledge. Only verified outcomes with evidence become knowledge (Article 13). Learning never grants authority and never weakens governance.

C.1 Daily operating rhythm

Eight hours. Brief, implement, assemble, test, attack, freeze.

The daily rhythm is how humans and agents share a clock without sharing authority. Architecture decides what should exist before agents generate. Evidence is frozen before the day closes.

Time | Activity | Hours | Purpose

- Time: 08:00--09:00
- Activity: Architecture & Command Briefing

- Hours: 1.0
- Purpose: Decide what should exist
- Time: 09:00--11:30
- Activity: AI Agent Implementation
- Hours: 2.5
- Purpose: Agents generate and modify systems
- Time: 11:30--13:00
- Activity: Assembly & Integration
- Hours: 1.5
- Purpose: Connect components
- Time: 14:00--15:30
- Activity: Testing & Validation
- Hours: 1.5
- Purpose: Prove functionality
- Time: 15:30--16:15
- Activity: Security & Governance Verification
- Hours: 0.75
- Purpose: Attack and verify controls
- Time: 16:15--17:00
- Activity: Evidence & Daily Freeze
- Hours: 0.75
- Purpose: Preserve decisions and evidence
- Time: TOTAL
- Activity:
- Hours: 8.0
- Purpose:

C.2 Weekly rhythm and six-day cycle

Heavy assembly, then integration, then attack. Every block: plan, assemble, test, attack, verify, freeze.

Weekly rhythm

Days | Mode

Monday--Thursday | Heavy assembly

Friday | Integration + testing

Saturday | Security + verification + documentation

Sunday | Light review / planning / recovery (2--4 hours)

Six-day work cycle

Each implementation block follows the same six beats. Skipping ATTACK or FREEZE is a process violation, not a schedule compression.

1. Plan
2. Assemble
3. Test
4. Attack
5. Verify

6. Freeze

C.3 Ten gates

Gate 1 constitutional runtime through Gate 10 red-team certification. Gates are evidence, not dates.

A gate is a certification event with an evidence package. Passing a calendar day does not pass a gate. Gate numbers are delivery checkpoints; they are not autonomy grants, not maturity levels, and not document-issue numbers.

Gate | Description | Blocks covered

Gate 1 | Constitutional Runtime works | Block 1

Gate 2 | Trust Kernel works | Block 2

Gate 3 | Authority / Capability security works | Block 3

Gate 4 | AI Agent runtime works | Block 4

Gate 5 | Product specification factory works | Block 5

Gate 6 | Controlled execution works | Block 6

Gate 7 | Independent verification works | Block 7

Gate 8 | Deployment lifecycle works | Block 8

Gate 9 | Multiple products remain isolated | Block 9

Gate 10 | Red-team certification passes | Block 10

C.4 Layer contracts

Purpose, permissions, and failure modes for every layer in the stack.

The thirteen-layer stack is the operative operating-model map of AGBOFA. No layer may assume the authority of a layer above it. Intelligence never is authority.

> Layer law. Each layer’s prohibitions are constitutional, not style. Autonomy grants named in a layer (A0, A1, A2) are agent autonomy, not enterprise maturity.

Human

Sovereign authority. Humans remain ultimately accountable for AGBOFA’s existence, operation, and major consequential actions.

Responsibilities

- Approve or reject all HIGH and CRITICAL risk actions.
- Hold emergency freeze authority at all times.
- Record overrides of agent recommendations with rationale.
- Approve constitutional amendments, autonomy increases, and gate certifications.
- Authorize recovery restore and system-wide freeze.

Inputs

- Agent proposals and recommendations.
- Risk classifications from the Trust Kernel.
- Evidence packages and gate certifications.
- Incident reports, freeze alerts, and isolation-breach alerts.

Outputs

- Human approvals and rejections.

- Emergency freeze orders.
- Override records with rationale.
- Constitutional amendment approvals.
- Autonomy grant and revocation decisions.

Permissions

- Emergency freeze.
- HIGH and CRITICAL approval.
- Constitutional amendment.
- Autonomy grants above A0.
- Recovery restore from backup.

Prohibited

- Delegating or diminishing human accountability (Article 1).
- Silent or unrecorded overrides of agent recommendations.
- Vacating freeze authority.
- Granting autonomy without evidence (Article 12).

Dependencies

- Constitution
- Governance
- Authority
- Audit

Data

- Human owner identity.
- Approval and rejection records.
- Override rationales.
- Freeze orders.
- Amendment approvals.

Failure modes

- No designated human owner.
- Freeze authority vacancy.
- HIGH or CRITICAL action proceeding without human approval.
- Unrecorded human override.

Security

- Human identity is verified independently of any agent.
- CRITICAL actions may require dual-control where policy so requires.
- Freeze capability is testable and always reachable.

Audit

- Every human decision is recorded: who, when, why, what was approved or rejected.
- Overrides are linked to the agent recommendation they supersede.
- Freeze and restore actions are hash-chained audit events.

Constitution

Supreme governing law of AGBOFA. No agent, model, subsystem, or process may violate it.

Responsibilities

- Define permanent architectural principles and the sixteen articles.
- Define autonomy levels A0, A1, and A2 and the evidence required to earn them.

- Define risk classifications CRITICAL, HIGH, MEDIUM, and LOW.
- Version constitutional text and bind it as machine-readable contracts.
- Reject any change that weakens governance (Article 11).

Inputs

- Human-approved amendment proposals.
- Constitutional review findings.
- Impact analyses from Architecture.

Outputs

- Versioned constitutional contracts.
- Invariant definitions consumed by the Trust Kernel.
- Autonomy and risk classification tables.
- Amendment history.

Permissions

- Declare law.
- Bind invariants that all lower layers must enforce.
- Require human approval for any amendment.

Prohibited

- Amendment without human approval.
- Amendment that weakens governance.
- Silent modification of articles, principles, or autonomy rules.
- Agent-originated constitutional change (Article 11).

Dependencies

- Human

Data

- Articles and principles.
- Autonomy level definitions (A0, A1, A2).
- Risk classification table.
- Amendment log and constitutional version.

Failure modes

- Constitution not loadable or not versioned.
- Invariant not encoded as a machine-readable contract.
- Drift between published text and enforced contracts.

Security

- Constitution repository is access-controlled and append-versioned.
- Only human-approved writes are accepted.
- Integrity of the constitutional freeze record is hash-verified.

Audit

- Every constitutional version is frozen with a hash and human sign-off.
- Amendment process steps are individually audited (proposal through evidence).

Governance

Policy and rule engine. Evaluates proposals against the Constitution before any authority is granted.

Responsibilities

- Evaluate every proposal for constitutional compliance and risk class.

- Route HIGH and CRITICAL actions to human approval.
- Version and store governance rules in the Constitution repository.
- Reject governance changes originating from governed agents (Article 11).
- Record every authority grant as an audit event (Article 2).

Inputs

- Proposals from Intelligence and Planning.
- Constitutional contracts and risk table.
- Human approvals and rejections.
- Evidence of prior governed operation for autonomy decisions.

Outputs

- Governance decisions (allow, deny, escalate).
- Risk classifications.
- Versioned policy records.
- Authority-grant requests to the Authority layer.

Permissions

- Classify risk.
- Evaluate policy.
- Request authority grants that originate from an authorized governance source.
- Escalate to Human.

Prohibited

- Granting authority that did not originate from an authorized governance source (Article 2).
- Accepting a governed agent’s self-modification of rules (Article 11).
- Auto-executing AI recommendations (Article 3).
- Default-allow (Article 4).

Dependencies

- Human
- Constitution

Data

- Policy definitions.
- Risk classification records.
- Governance decision log.
- Rule versions.

Failure modes

- Policy engine unavailable.
- Proposal advancing without a governance decision.
- Rule change without human approval.

Security

- Governed agents cannot write governance rules that apply to themselves.
- Policy evaluation is deterministic and replayable from audit.
- Unauthorized governance sources are rejected by the Authority Kernel.

Audit

- Every proposal evaluation is an audit event: policy version, risk class, decision, authority requested.
- Governance rule versions are hash-chained.

Authority

Explicit grant system. Every consequential action requires an identifiable, traceable authority source.

Responsibilities

- Issue, delegate, and revoke authority grants.
- Maintain authority chains traceable to a human or constitutional source.
- Bind each grant to scope, duration, and revocation conditions (Article 4).
- Reject implicit authority, default-allow, and self-grants (Articles 2 and 4).
- Refuse any grant that does not originate from an authorized governance source.

Inputs

- Governance decisions.
- Human approvals.
- Revocation requests from Security, Trust Kernel, or Human.
- Expiry and scope of existing grants.

Outputs

- Authority chains attached to actions.
- Grant, delegation, and revocation records.
- Rejection of invalid grants.

Permissions

- Issue scoped authority.
- Delegate within an existing grant’s bounds.
- Revoke any grant.

Prohibited

- Implicit authority.
- "The agent knew what to do" as an authority source.
- Default allow.
- Agent self-grant.
- External system directly granting authority inside AGBOFA.

Dependencies

- Human
- Constitution
- Governance

Data

- Authority grants.
- Delegation graph.
- Revocation list.
- Authority chain snapshots.

Failure modes

- Action presented without an authority chain.
- Chain that does not trace to a human or constitutional source.
- Grant past its duration or outside its scope.

Security

- Authority Kernel rejects grants from unauthorized sources.
- Chains are tamper-evident and stored with the audit event.
- Revocation is immediate and globally visible to Capability and Execution.

Audit

- Every grant, delegation, and revocation is an audit event.

- Every consequential action records its full authority chain.

Capabilities

Scoped token system. An actor can only perform actions for which it possesses the required capability.

Responsibilities

- Issue short-lived capability tokens with explicit scope, environment, and expiry (Article 5).
- Validate tokens on every execution attempt.
- Reject missing, expired, or out-of-scope tokens.
- Refuse capability reuse across product boundaries by default (Article 14).
- Revoke tokens on abuse, isolation breach, or autonomy reduction.

Inputs

- Valid authority chains.
- Requested capability name, product, environment, and resource.
- Constraints (max duration, max operations, allowed commands).

Outputs

- Capability tokens.
- Validation results.
- Revocation records.
- Scope-violation rejections.

Permissions

- Issue tokens only against a valid authority chain.
- Validate and expire tokens.
- Revoke tokens.

Prohibited

- Broad capabilities such as "agent has access to everything".
- Capability escalation without governance approval.
- Capability reuse across product boundaries by default.
- Renewal except through governance.

Dependencies

- Authority
- Governance
- Constitution

Data

- Token records (tokenId, capability, scope, grantedTo, grantedBy, issuedAt, expiresAt, constraints).
- Revocation list.
- Scope catalogue.

Failure modes

- Execution attempted with missing, expired, or out-of-scope token.
- Token forged or replayed.
- Cross-product token accepted.

Security

- Tokens are unforgeable, short-lived, and bound to agent, product, and environment.
- Isolation Guard consults token scope before any cross-boundary access.
- Forged or escalated tokens are blocked and audited (Gate 3).

Audit

- Issuance, validation, rejection, and revocation are audit events.
- Each execution records the tokenId and scope used.

Intelligence

AI provider abstraction. Intelligence proposes; it never is authority (Article 3, Article 16).

Responsibilities

- Expose all model access through the Intelligence Contract.
- Keep provider adapters pluggable and versioned.
- Admit every AI output as a PROPOSAL, never as a command.
- Make the distinction between intelligence and authority explicit in every agent interaction.
- Refuse direct provider access that bypasses the abstraction layer.

Inputs

- Agent context and tasks from the Agent Runtime.
- Provider-agnostic contract calls.
- Constitutional constraints (no training on data, isolation, audit logging).

Outputs

- Recommendations, plans, and generated artifacts labeled as PROPOSALS.
- Declared confidence that never substitutes for authority.
- Provider-adapter health and contract-version records.

Permissions

- Call providers only through the Intelligence Contract.
- Produce proposals.
- Declare confidence.

Prohibited

- Treating AI outputs as commands.
- Auto-executing recommendations without governance evaluation.
- Substituting AI confidence for authority.
- Hard-coding provider-specific APIs in core AGBOFA components.
- Treating any provider as irreplaceable.

Dependencies

- Constitution
- Governance
- Authority
- Capabilities

Data

- Intelligence Contract records.
- Provider adapter versions.
- Agent registry and roles.
- Proposal objects.

Failure modes

- Direct provider call bypassing the contract.
- Proposal emitted without an authority-chain placeholder (must be filled by Governance).
- Core component depending on a provider-specific feature.

Security

- Data isolation and no-training-on-data flags are contract-enforced.
- Provider switching does not require core changes.
- Prompt-injection handling is a verification concern, not an authority grant.

Audit

- Every contract call records provider, contract version, agent, task, and that the output is a proposal.
- Direct provider-access attempts are architecture-violation flags (Article 16).

Planning

Governed action plans. Turns approved intent into sequenced, classifiable actions -- still proposals until governance and authority attach.

Responsibilities

- Produce implementation plans from approved blueprints and architecture.
- Classify each planned action and attach intended risk class.
- Sequence work so verification and evidence gates are explicit steps, not afterthoughts.
- Hand plans to Governance before any execution is requested.
- Never treat a plan as an authority source (Article 4).

Inputs

- Approved product specifications, blueprints, and architecture proposals.
- Requirements, domain models, and Product Passport references.
- Policy and risk tables.

Outputs

- Governed action plans.
- Per-step action classifications.
- Declared verification and evidence requirements per step.

Permissions

- Draft plans.
- Propose action sequences.
- Attach intended risk and verification requirements.

Prohibited

- Executing a plan.
- Using the plan itself as an authority source.
- Omitting verification steps for consequential actions (Article 7).
- Planning cross-tenant or cross-environment work without explicit authorization (Article 14).

Dependencies

- Intelligence
- Governance
- Constitution
- Capabilities

Data

- Action plans.
- Step classifications.
- Verification requirements per step.
- Plan versions.

Failure modes

- Plan advancing to execution without governance evaluation.

- Consequential step without a verification requirement.
- Plan that assumes implicit authority.

Security

- Plans are scoped to a product and environment.
- Plan objects cannot carry capability tokens; tokens are issued later against authority.

Audit

- Plan creation, version, and governance evaluation are audit events.
- Each planned consequential step records its intended authority and verification path.

Execution

Isolated broker and workspace. Intelligence never receives unrestricted execution access (Article 6).

Responsibilities

- Admit execution only through the Execution Broker.
- Allocate isolated workspaces with resource limits, timeouts, and failure handling.
- Require a valid authority chain and an in-scope capability token before running anything.
- Reject any direct execution attempt that bypasses the governance chain.
- Return results as claims that still require independent verification (Article 7).

Inputs

- Governance-approved actions.
- Authority chains.
- Capability tokens.
- Human approvals where risk class requires them.

Outputs

- Workspace-scoped results.
- Execution status (success, failure, rejected, error).
- Resource-usage and timeout records.

Permissions

- Run approved actions inside isolated workspaces.
- Enforce limits and timeouts.
- Quarantine an agent that attempts bypass.

Prohibited

- Direct agent access to production systems.
- Execution outside isolated workspaces.
- Bypassing the Execution Broker.
- Treating execution as proof of correctness (Article 7).

Dependencies

- Authority
- Capabilities
- Governance
- Planning

Data

- Workspace records.
- Worker assignments.
- Execution results.
- Limit and timeout configurations.

Failure modes

- Bypass attempt.
- Workspace escape.
- Runaway agent (rate limit must fire).
- Result promoted without verification.

Security

- Workspaces are tenant-, product-, and environment-scoped.
- Production credentials are never issued to agents as broad capabilities.
- Quarantine isolates a violating agent from further execution.

Audit

- Every execution records actor, authority, capability, product, environment, action, result.
- Bypass and quarantine events are security alerts.

Verification

Independent validation. Important outcomes must be verified by a component that did not perform the action (Articles 7 and 8).

Responsibilities

- Verify functional correctness and constitutional compliance.
- Run dedicated verification for builds, tests, deployments, and data operations.
- Produce evidence that is stored and auditable.
- Block outcomes that lack independent verification from advancing.
- Flag consequential actions that have no verification evidence.

Inputs

- Execution results.
- Declared verification requirements from Planning.
- Constitutional invariants.
- Test, build, security, and deployment artifacts.

Outputs

- Verification results (pass, fail, unverified).
- Evidence packages at the required evidence level (E1--E4).
- Promotion or block decisions for the next lifecycle stage.

Permissions

- Read execution artifacts within the same product and environment scope.
- Invoke independent test, build, and compliance checkers.
- Mark outcomes unverified and block promotion.

Prohibited

- Agents verifying their own work.
- Verification by the same component that performed the action.
- Verification that only checks surface-level success.
- Skipping verification for consequential actions.

Dependencies

- Execution
- Constitution
- Evidence

Data

- Verification results.
- Evidence-level assignments (E0 is unacceptable).
- Block and re-execution records.

Failure modes

- Self-verification accepted.
- Surface-level-only check treated as independent verification.
- Consequential action lacking verification evidence.

Security

- Verifier identity is distinct from executor identity.
- Verification agents operate at autonomy A2 only within pre-approved verification scope.
- Isolation Guard still applies to verifier access.

Audit

- Every verification records method, result, verifier identity, and evidenceId.
- Unverified consequential actions are flagged and cannot be used for decision-making.

Evidence

Immutable proof store. Claims about system state require evidence; trust is earned, not assumed (Article 9).

Responsibilities

- Store evidence with provenance for every consequential claim.
- Prefer machine-verifiable evidence.
- Hash evidence objects and bind them to the originating action and verification.
- Refuse claims that lack evidence; mark them unverified.
- Never treat absence of errors as evidence of correctness.

Inputs

- Verification results.
- Test outputs, configuration snapshots, security scans, independent-verification packages.
- Provenance (actor, action, product, environment, timestamp).

Outputs

- Evidence records with hashes and provenance.
- Unverified-claim markers.
- Evidence packages referenced by audit events.

Permissions

- Append evidence.
- Validate evidence hashes.
- Serve evidence to Audit, Memory, and Human review.

Prohibited

- Accepting agent assertions without evidence.
- Treating absence of errors as evidence of correctness.
- Storing claims without supporting evidence.
- Silent modification of stored evidence.

Dependencies

- Verification
- Audit

Data

- Evidence objects.
- Hashes and provenance.
- Evidence-level tags (E1--E4).
- Links to verification and audit events.

Failure modes

- Claim used for decision-making without evidence.
- Evidence hash mismatch.
- E0 (agent assertion only) accepted for any consequential claim.

Security

- Evidence store is append-oriented and integrity-checked.
- Write access is limited to Verification and designated collectors.
- Tamper attempts escalate as security alerts.

Audit

- Every evidence write records who, what action, what was verified, and the hash.
- EvidenceIds appear on the corresponding audit events.

Audit

Append-only, tamper-evident trail. Consequential actions create durable audit records (Article 10).

Responsibilities

- Generate an audit event for every consequential action.
- Keep the log append-only and hash-chained.
- Record who, which agent, which authority, which capability, which product, which environment, what action, when, why, what happened, what was verified, and what evidence exists.
- Detect tampering with timestamps, hashes, or chains and trigger a security alert.
- Refuse any audit bypass.

Inputs

- Events from every layer: Human, Governance, Authority, Capabilities, Execution, Verification, Evidence, Recovery.
- Previous hash for chain continuity.

Outputs

- Append-only audit events.
- Tamper-detection alerts.
- Queryable trail for Human, Compliance, and Recovery.

Permissions

- Append events.
- Hash-chain events.
- Detect and alert on tamper.

Prohibited

- Audit record modification or deletion.
- Audit bypass for any reason.
- Tampering with audit timestamps, hashes, or chains.

Dependencies

- Constitution
- Evidence

Data

- AuditEvent records (eventId, timestamp, actor, action, authority, capability, target, result, verification, hash, previousHash).
- Hash chain.
- Tamper alerts.

Failure modes

- Missing audit event for a consequential action.
- Broken hash chain.
- Detected modification or deletion.

Security

- Write path is append-only at the storage layer.
- Tamper detection triggers system freeze and security investigation (Article 10).
- Read access is governed; write access is not available to governed agents.

Audit

- The audit layer audits itself: chain verification jobs and freeze events are themselves audit events.

Memory

Verified knowledge only. AI saying something does not become AGBOFA knowledge (Article 13).

Responsibilities

- Accept knowledge candidates only after verification evidence exists.
- Link every stored knowledge item to its evidence and originating verified outcome.
- Version knowledge and keep it auditable.
- Reject unverified claims and AI-confidence-as-validity.
- Serve verified knowledge to future decision support -- never as authority.

Inputs

- Verified outcomes from Verification.
- Evidence records from Evidence.
- Knowledge-candidate proposals from Intelligence and Learning.

Outputs

- Versioned institutional knowledge items.
- Rejections of unverified candidates.
- Knowledge references for Planning and Human review.

Permissions

- Store verified knowledge.
- Reject unverified candidates.
- Version and retrieve knowledge.

Prohibited

- Direct promotion of AI outputs to knowledge without verification.
- Treating AI confidence as knowledge validity.
- Storing unverified claims in institutional memory.

Dependencies

- Verification
- Evidence
- Audit

- Constitution

Data

- Knowledge candidates.
- Validated knowledge items with evidence links.
- Knowledge versions.

Failure modes

- Unverified candidate stored.
- Knowledge item without an evidence link.
- Confidence score used as a validity bit.

Security

- Write path requires a verification evidenceId.
- Knowledge is tenant- and product-scoped unless explicitly authorized otherwise.

Audit

- Every accept and reject of a knowledge candidate is an audit event.
- Knowledge versions record evidenceId and verifier.

Learning

Institutional growth from verified outcomes only. Learning never grants authority or weakens governance.

Responsibilities

- Propose improvements from verified outcomes stored in Memory.
- Feed future Planning and Human briefing -- as proposals, not commands.
- Never convert an AI assumption into institutional knowledge merely because it was generated (Article 13).
- Never increase autonomy from self-assessment (Article 12).
- Subject learning-driven change to the same governance, verification, and audit path as any other change.

Inputs

- Institutional knowledge from Memory.
- Verified outcomes and evidence.
- Operational metrics that have passed verification.

Outputs

- Improvement proposals.
- Knowledge-candidate submissions back to Memory (still requiring verification).
- Briefings for Human and Planning.

Permissions

- Propose improvements.
- Submit knowledge candidates.
- Summarize verified history.

Prohibited

- Auto-executing learned changes.
- Autonomy increase from learning alone.
- Writing governance rules or constitutional text.
- Bypassing verification because a pattern ’has been seen before’.

Dependencies

- Memory
- Evidence

- Verification
- Governance
- Intelligence

Data

- Improvement proposals.
- Learning-cycle records linked to verified outcomes.
- Rejected unverified generalizations.

Failure modes

- Learned change applied without governance.
- Unverified generalization stored as knowledge.
- Self-assessed autonomy increase.

Security

- Learning has no execution capability of its own.
- Outputs re-enter the system as PROPOSALS (Article 3).

Audit

- Every learning proposal records the verified outcomes it cites.
- Rejected unverified generalizations are audited so they are not retried as facts.

C.5 Agent catalogue

Named agents, tools, prohibitions, and autonomy grants A0 / A1 / A2.

Every named agent is a role under the Intelligence Contract. Outputs are proposals unless a scoped A2 grant (Verification only, in this catalogue) authorizes governed execution of independent verification.

> Autonomy in this catalogue. A0 advisory intelligence. A1 proposal. A2 governed execution of independent verification within pre-approved scope. No agent in this catalogue is A3 or above. Confidence never substitutes for authority (Article 3).

Product Discovery (A1)

Transform ideas into requirements.

Authority. A1 -- Proposal. Submit formal discovery proposals for governance evaluation. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Product idea or intake brief.
- Human owner intent.
- Existing Product Passport references where a product already exists.

Outputs

- Discovery report labeled as a PROPOSAL.
- Draft requirements candidates.
- Assumption log.

Tools

- Discovery workflow.
- Intake forms.
- Intelligence Contract (reasoning, analysis).

Prohibited

- Executing implementation work.
- Treating discovery output as a command (Article 3).
- Self-granting authority (Article 2).
- Crossing tenant or product boundaries by default (Article 14).

Requirements (A1)

Define and validate requirements.

Authority. A1 -- Proposal. Submit formal requirements for governance evaluation. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Discovery report.
- Human-approved intake.
- Domain findings.
- Constitutional risk table.

Outputs

- Classified requirements.
- Assumption tracking records.
- Requirements proposals for Product Passport and Blueprint.

Tools

- Requirements engine.
- Classification and assumption tracking.
- Intelligence Contract (analysis, planning).

Prohibited

- Promoting requirements to knowledge without verification (Article 13).
- Auto-executing a requirement as work.
- Omitting risk classification for consequential requirements.
- Self-granting authority.

Domain Analyst (A1)

Analyze domain models.

Authority. A1 -- Proposal. Submit domain-model proposals for governance evaluation. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved requirements.
- Existing domain entities (Product, Tenant, User, Agent, Capability, Authority, Action, Policy, Evidence, AuditEvent, Execution, Verification, Environment).
- Product Passport drafts.

Outputs

- Domain model proposals.
- Entity and boundary maps.
- Isolation notes for tenant, product, and environment.

Tools

- Domain modeling.

- Entity extraction.
- Intelligence Contract (analysis).

Prohibited

- Writing production schema.
- Collapsing tenant or product boundaries.
- Treating a domain model as authority to execute.
- Self-granting authority.

Architecture (A1)

Propose system architecture.

Authority. A1 -- Proposal. Submit architecture and contract proposals for governance evaluation. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved requirements and domain model.
- Product Blueprint.
- Constitutional contracts and Intelligence Contract.
- Amendment impact-analysis requests.

Outputs

- Architecture proposals.
- Contract schemas (identity, authority, capability, action, governance, verification, evidence, audit).
- Impact analyses for constitutional change.

Tools

- Architecture proposal system.
- Contract generation.
- Intelligence Contract (reasoning, planning).

Prohibited

- Hard-coding provider-specific APIs in core components (Article 16).
- Implementing architecture without human approval of the proposal.
- Weakening governance in a proposed design (Article 11).
- Self-granting authority.

Database (A1)

Design and manage data models.

Authority. A1 -- Proposal. Submit schema and migration proposals for governance evaluation. No production data-plane execution.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved domain model.
- Architecture proposal.
- Existing migrations and validation schemas.

Outputs

- Schema and migration proposals.
- Zod/validation schema proposals.

- Data-isolation notes (tenant, product, environment).

Tools

- Schema generation.
- Migration drafting.
- Intelligence Contract (code generation, analysis).

Prohibited

- Applying migrations to production except through Execution Broker after approval.
- Cross-tenant schema or data access (Article 14).
- Recovery procedures that bypass governance (Article 15).
- Self-granting authority.

UX (A1)

Design user experience.

Authority. A1 -- Proposal. Submit UX proposals for governance evaluation. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved requirements and blueprint.
- Product Passport.
- Human design constraints.

Outputs

- UX design proposals.
- Interaction and information-architecture specs.
- Accessibility and evidence notes for user-facing flows.

Tools

- Design specifications.
- Interaction modeling.
- Intelligence Contract (analysis, code generation for mock specs only as proposals).

Prohibited

- Shipping UI to production without governance, verification, and deployment approval.
- Collecting or exposing cross-tenant user data.
- Self-granting authority.

Security (A1)

Identify and address security concerns.

Authority. A1 -- Proposal. Submit security findings, classifications, and revocation recommendations. No silent production change.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Architecture, capability design, and execution flows.
- Action-classification policy.
- Red-team scenarios (privilege escalation, tenant escape, injection, audit tampering).

Outputs

- Security findings and threat models as PROPOSALS.

- Classification-logic proposals.
- Attack-simulation reports (evidence, not self-certified pass).
- Capability-revocation recommendations.

Tools

- Threat modeling.
- Attack simulation (governed).
- Classification review.
- Intelligence Contract (analysis).

Prohibited

- Granting or escalating capabilities.
- Bypassing the Execution Broker to 'test' production.
- Modifying audit records (Article 10).
- Self-granting authority.
- Verifying its own remediations (Article 8).

Repository (A1)

Manage code repositories.

Authority. A1 -- Proposal. Submit repository and code-change proposals. Execution only through the Execution Broker under a scoped token.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved architecture, schema, and implementation plans.
- Capability tokens scoped to a product repository and environment.
- Human and governance approvals for writes.

Outputs

- Repository-change proposals and, when authorized, governed patches via Execution Broker.
- Workspace file-operation results (still unverified until Verification runs).

Tools

- Repository operations (governed).
- Workspace file operations via Execution Broker.
- Intelligence Contract (code generation).

Prohibited

- Direct access to production systems (Article 6).
- Execution outside isolated workspaces.
- Bypassing the Execution Broker.
- Cross-product repository access by default (Article 14).
- Self-granting authority or capability escalation (Article 5).

QA (A1)

Design and execute tests.

Authority. A1 -- Proposal. Submit test designs and request governed test execution. QA execution is not independent verification (Article 8).

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Approved requirements, architecture, and implementation artifacts.
- Constitutional invariant list.
- Test-scope capability tokens.

Outputs

- Test-design proposals.
- Governed test-run results (not independent verification).
- Coverage notes against invariants and gates.

Tools

- Test generation.
- Unit, integration, and E2E runners via Execution Broker.
- Intelligence Contract (code generation, analysis).

Prohibited

- Serving as independent verification of work the same agent designed or ran as the executor.
- Treating a green suite as evidence of constitutional compliance without the Verification Agent.
- Skipping tests for consequential actions (Article 7).
- Self-granting authority.

Verification (A2)

Independently verify outcomes.

Authority. A2 -- Governed execution of independent verification within pre-approved scope, with governance checks. No implementation execution.

Confidence. Declared on every output; never a substitute for authority or for evidence (Articles 3 and 9).

Inputs

- Execution results from other agents.
- Declared verification requirements.
- Constitutional invariants.
- Candidate evidence (tests, configs, scans, deployment artifacts).

Outputs

- Independent verification results.
- Evidence packages (E1--E4 as required by risk class).
- Gate certifications.
- Unverified flags and promotion blocks.

Tools

- Verification engine.
- Independent test, build, deployment, and compliance checkers.
- Evidence collection.
- Intelligence Contract (analysis only; outputs remain proposals until verification records are written).

Prohibited

- Verifying work this agent performed (Article 8).
- Surface-level-only checks presented as independent verification.
- Implementing product features.
- Promoting unverified outcomes.
- Self-granting authority or raising its own autonomy (Article 12).

Deployment (A1)

Manage deployments.

Authority. A1 -- Proposal. Submit deployment plans and request governed execution. Production deploy requires human approval (HIGH/CRITICAL).

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Verified build artifacts.
- Deployment approval (human for HIGH/CRITICAL).
- Environment references from the Product Passport.
- Scoped capability tokens for the target environment.

Outputs

- Deployment proposals and governed deployment requests.
- Environment-change records.
- Post-deployment verification requests to the Verification Agent.

Tools

- Deployment system (via Execution Broker).
- Environment management (scoped).
- Intelligence Contract (planning).

Prohibited

- Deploying without verification evidence (Articles 7 and 8).
- Deploying to an environment outside token scope.
- Skipping post-deployment verification.
- Cross-environment capability use by default (Article 14).
- Self-granting authority.

Operations (A1)

Monitor and manage operations.

Authority. A1 -- Proposal. Submit operational and incident proposals. Freeze, rollback, and restore require authorized recovery paths (Article 15).

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Running-product telemetry (scoped).
- Incident reports.
- Compliance and lifecycle state from the Product Passport.
- Recovery-procedure definitions.

Outputs

- Operational briefings and incident proposals.
- Monitoring and compliance observations as PROPOSALS.
- Escalations for freeze, rollback, or restore (do not execute CRITICAL recovery unaided).

Tools

- Monitoring.
- Incident workflow.
- Compliance observations.

- Intelligence Contract (analysis).

Prohibited

- Unaudited recovery actions (Article 15).
- Recovery that bypasses governance.
- Cross-tenant operational access by default.
- Self-granting authority.
- Treating absence of alerts as evidence of correctness (Article 9).

Financial Intelligence (A0)

Advise on cost, budget, and financial risk of governed product work. Intelligence only -- not authority.

Authority. A0 -- Advisory. Recommendations only. No execution authority and no proposal-execution path of its own.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Product Passport and plan estimates.
- Verified operational usage where available.
- Human financial constraints.

Outputs

- Cost and budget recommendations as PROPOSALS.
- Financial-risk notes attached to plans.

Tools

- Cost analysis.
- Budget modeling.
- Intelligence Contract (analysis).

Prohibited

- Executing spend or changing billing configuration.
- Treating financial advice as an authority source (Article 3).
- Auto-executing recommendations.
- Self-granting authority or autonomy (Article 12).
- Storing unverified financial claims as institutional knowledge (Article 13).

Legal/Compliance Intelligence (A0)

Advise on legal, policy, and compliance posture against constitutional and external frameworks. Intelligence only -- not authority.

Authority. A0 -- Advisory. Recommendations only. No execution authority. Cannot amend governance or the Constitution.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Constitutional articles and policies.
- Product requirements and data-handling designs.
- Audit and evidence summaries.

Outputs

- Compliance gap analyses as PROPOSALS.
- Policy-mapping recommendations.

- Escalations to Human for HIGH/CRITICAL legal risk.

Tools

- Policy mapping.
- Compliance checklists.
- Intelligence Contract (analysis).

Prohibited

- Amending the Constitution or governance rules (Article 11).
- Certifying compliance without independent verification and evidence.
- Auto-executing remediations.
- Self-granting authority.

Research Intelligence (A0)

Research options, prior art, and comparative approaches to inform proposals. Intelligence only -- not authority.

Authority. A0 -- Advisory. Recommendations only. No execution authority.

Confidence. Declared on every output; never a substitute for authority or for evidence (Articles 3 and 9).

Inputs

- Research questions from Human or other agents' proposals.
- Verified institutional knowledge from Memory.
- Declared unknowns and assumptions from Requirements.

Outputs

- Research briefs as PROPOSALS.
- Sourced comparisons.
- Knowledge candidates that still require verification before Memory accepts them.

Tools

- Literature and comparative analysis.
- Intelligence Contract (reasoning, analysis).

Prohibited

- Writing unverified claims into Memory (Article 13).
- Treating research as a command or as architecture approval.
- Auto-executing recommended tools or vendors.
- Self-granting authority.

Performance Intelligence (A0)

Advise on performance characteristics, capacity, and optimization. Intelligence only -- not authority.

Authority. A0 -- Advisory. Recommendations only. No execution authority.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Verified performance measurements where they exist.
- Architecture and deployment proposals.
- Declared performance requirements.

Outputs

- Performance analyses as PROPOSALS.
- Optimization recommendations.

- Requests for governed performance-test execution (does not run them as independent verification).

Tools

- Performance analysis.
- Capacity modeling.
- Intelligence Contract (analysis).

Prohibited

- Changing production configuration to 'tune' performance.
- Treating absence of slowdown reports as evidence of correctness (Article 9).
- Self-verifying its own optimization claims (Article 8).
- Self-granting authority.

Reliability Intelligence (A0)

Advise on reliability, SLOs, failure modes, and recovery readiness. Intelligence only -- not authority.

Authority. A0 -- Advisory. Recommendations only. No execution authority. Cannot freeze, roll back, or restore.

Confidence. Declared on every output; never a substitute for authority (Article 3).

Inputs

- Incident history (audit-backed).
- Recovery-procedure definitions.
- Verified operational outcomes.
- Architecture proposals.

Outputs

- Reliability and SLO recommendations as PROPOSALS.
- Failure-mode analyses.
- Recovery-test recommendations for Human and Recovery System.

Tools

- SLO/SLI analysis.
- Failure-mode analysis.
- Intelligence Contract (analysis).

Prohibited

- Executing freeze, rollback, or restore (Article 15 -- recovery is constitutional and authorized separately).
- Unaudited recovery advice applied as action.
- Operating-without-recovery-procedures recommendations that weaken governance.
- Self-granting authority.

D.1 Repository structure

Constitution at the root. Core, intelligence, factory, products, operations, evidence, command center.

The repository is itself a constitutional artifact. The Constitution directory is supreme. Products never import core internals that would let an agent self-grant authority.

```
agbofa/
??? constitution/              # THE CONSTITUTION (supreme)
?   ??? articles/              # Each article as machine-readable contract
?   ??? principles/            # Permanent architectural principles
?   ??? amendments/            # Constitutional amendment process
?   ??? version-control/       # Constitutional versioning
?
??? core/                      # AGBOFA CORE PLATFORM
```

```
?   ??? trust-kernel/           # Auth, governance, policy engine
?   ??? authority/              # Authority model, delegation, revocation
?   ??? capability/             # Capability tokens, scopes, expiry
?   ??? execution/              # Broker, workspace, workers, sandbox
?   ??? verification/           # Independent verification engine
?   ??? evidence/               # Evidence collection, hashing, storage
?   ??? audit/                  # Append-only audit log
?   ??? recovery/               # Freeze, rollback, restore
?
??? intelligence/               # AI ABSTRACTION LAYER
?   ??? contract/               # Intelligence Contract (provider-agnostic)
?   ??? adapters/               # Provider adapters
?   ??? runtime/                # Agent runtime, context, task management
?   ??? memory/                 # Institutional memory and learning
?
??? factory/                    # PRODUCT FACTORY
?   ??? discovery/              # Product discovery workflow
?   ??? requirements/           # Requirements engine
?   ??? blueprint/              # Product blueprint generation
?   ??? architecture/           # Architecture proposal system
?   ??? planning/               # Implementation planning
?
??? products/                   # PRODUCTS BUILT BY AGBOFA
?   ??? product-a/
?   ?   ??? passport.json       # Product Passport (identity contract)
?   ?   ??? requirements/
?   ?   ??? blueprint/
?   ?   ??? source/
?   ?   ??? evidence/
?   ?   ??? audit/
?   ??? product-b/
?
??? operations/                 # ENTERPRISE OPERATIONS
?   ??? monitoring/
?   ??? incident/
?   ??? deployment/
?   ??? compliance/
?
??? templates/                  # TECHNOLOGY TEMPLATES
?   ??? web-app/
?   ??? api-service/
?   ??? database/
?
??? evidence/                   # AGBOFA'S OWN EVIDENCE
?   ??? daily-freezes/
?   ??? gate-certifications/
?   ??? red-team/
?   ??? completion-matrix.json
?
??? command-center/             # AGBOFA COMMAND CENTER
    ??? api/
    ??? dashboard/
    ??? cli/
```

D.2 Technology stack

TypeScript, Node 20, Postgres 16, SQLite, Drizzle, BullMQ, XState 5, Zod, Vitest, Playwright, Fastify, JWT and capability tokens, Docker Compose (phase 1), React and Vite.

The stack is chosen for type safety, explicit state, and auditable storage. Provider-specific AI SDKs do not appear in core. Phase 1 deployment is a single-node Docker Compose so the execution path remains inspectable.

Component | Technology | Rationale

Language | TypeScript (strict mode) | Type safety, ecosystem maturity

Runtime | Node.js 20+ | Production-ready, widespread support

Database (Primary) | PostgreSQL 16 | Enterprise-grade, ACID compliance

Database (Isolated) | SQLite | Lightweight test and workspace environments

ORM | Drizzle ORM | SQL-first, TypeScript-native, lightweight

Queue | BullMQ (Redis-based) | Production-ready job queue

State management | XState 5 | Explicit state machines, TypeScript-native

Validation | Zod | Runtime validation, TypeScript inference

Testing (Unit / Integration) | Vitest | Fast, modern, TypeScript-native

Testing (E2E) | Playwright | Browser automation, cross-browser

API framework | Fastify | High-performance, TypeScript-native

Authentication | Custom JWT + capability tokens | Constitutional authority model

Audit storage | PostgreSQL + append-only tables | Durable, queryable, hash-chained

Containerization | Docker + Docker Compose | Consistent environments

Deployment (Phase 1) | Single-node Docker Compose | Controlled, auditable

CLI | Commander.js | Mature CLI framework

Dashboard | React + Vite | Command center UI

Documentation | Markdown + OpenAPI 3.1 | Machine-readable API docs

D.3 Product Passport

Identity contract for every product. v5.1 binds constitutionVersion and a closed lifecycleState union.

Every product carries a Product Passport. The operative issue adds constitutionVersion so a product is bound to a constitutional freeze, and closes lifecycleState to the factory union. An unknown state is a defect, not a feature.

> Closed lifecycle. lifecycleState is a union of factory stages, not a free string. constitutionVersion records which constitutional issue the product is governed by. Neither field is an autonomy grant or a maturity certification.

```
interface ProductPassport {
  productId: string;
  productName: string;
  tenantId: string;
  ownerId: string;
  version: string;
  constitutionVersion: string;
  lifecycleState:
    | "idea"
    | "discovery"
    | "requirements"
    | "domain_model"
    | "passport"
    | "blueprint"
    | "architecture"
    | "human_approval"
    | "implementation"
    | "execution"
    | "build"
    | "test"
    | "security"
    | "verification"
    | "evidence"
    | "deployment_approval"
    | "deployment"
    | "post_deployment_verification"
    | "operations"
```

```
    | "learning"
    | "improvement"
    | "retirement";
  requirements: RequirementReference[];
  architecture: ArchitectureReference;
  technologyProfile: TechnologyProfile;
  database: DatabaseReference;
  repositories: RepositoryReference[];
  environments: EnvironmentReference[];
  deployments: DeploymentReference[];
  agents: AgentReference[];
  capabilities: CapabilityReference[];
  policies: PolicyReference[];
  auditHistory: AuditReference[];
  evidence: EvidenceReference[];
  incidents: IncidentReference[];
  recoveryState: RecoveryReference;
}
```

D.4 Capability token and audit event

Scoped, short-lived capability. Append-only, hash-chained audit.

Capability token

An actor can only perform actions for which it possesses the required capability (Article 5). Tokens are short-lived, scoped to product and environment, and renewable only through governance.

```
interface CapabilityToken {
  tokenId: string;
  capability: string;          // e.g., "repo.read"
  scope: {
    productId: string;
    environment: string;
    resource?: string;
  };
  grantedTo: {
    agentId: string;
    role: string;
  };
  grantedBy: {
    authorityChain: string[];
    humanApproval?: string;
  };
  issuedAt: string;
  expiresAt: string;
  constraints: {
    maxDuration?: number;
    maxOperations?: number;
    allowedCommands?: string[];
  };
}
```

Audit event

Consequential actions create durable audit records (Article 10). Events are append-only and hash-chained. Modification, deletion, or bypass is a security incident.

```
interface AuditEvent {
  eventId: string;
  timestamp: string;
  actor: {
    type: "human" | "agent" | "system";
    id: string;
```

```
    role?: string;
  };
  action: {
    type: string;
    classification: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    description: string;
  };
  authority: {
    chain: string[];
    token?: string;
    humanApproval?: string;
  };
  capability: {
    name: string;
    scope: string;
    tokenId?: string;
  };
  target: {
    productId?: string;
    environment?: string;
    resource?: string;
  };
  result: {
    status: "success" | "failure" | "rejected" | "error";
    details?: string;
  };
  verification: {
    performed: boolean;
    method?: string;
    result?: string;
    evidenceId?: string;
  };
  hash: string;
  previousHash: string;
}
```

D.5 Intelligence Contract

Provider-agnostic contract. Adapters for Claude, GPT, and Local are replaceable.

All intelligence access flows through the Intelligence Contract (Article 16). The contract names capabilities and constraints, not a vendor. Direct provider access from core is an architecture violation.

```
interface IntelligenceContract {
  contractId: string;
  version: string;
  provider: string;
  capabilities: {
    reasoning: boolean;
    codeGeneration: boolean;
    analysis: boolean;
    planning: boolean;
    toolUse: boolean;
  };
  constraints: {
    maxContextLength: number;
    maxOutputLength: number;
    timeoutMs: number;
    retryPolicy: RetryPolicy;
  };
  security: {
    dataIsolation: boolean;
    noTrainingOnData: boolean;
    auditLogging: boolean;
  };
}
```

Adapter path

Claude, GPT, and Local are adapters behind the contract. Naming them does not make them constitutional. Any adapter is replaceable without a core change. Switching adapters does not grant autonomy and does not raise maturity.

1. Agent task (proposal context)
2. Intelligence Contract
3. Adapter: Claude
4. Adapter: GPT
5. Adapter: Local
6. Provider runtime
7. Contracted output (PROPOSAL)
8. Governance evaluation

> No provider is irreplaceable. Hard-coding provider-specific APIs in core AGBOFA components is prohibited. Provider switching is possible without core changes. The Execution Broker still rejects any action whose authority chain traces only to a model.

E.1 Enterprise maturity

Platform capability scale. The freeze wrote L0--L5. The operative issue writes M0--M5. M is not A.

Enterprise maturity is the M scale. It describes what the platform can do, not what an individual agent may do. Day-60 certification is not a claim of M5.

> M is not A. M0--M5 are platform maturity. A0--A5 are agent autonomy grants. Reaching M2 does not raise any agent to A2. Granting A2 to the Verification Agent does not certify M2. Mixing them is the error the received freeze's single L ladder produced.

Level | Name | Description | Key capabilities | v5.0 wrote

- Level: M0
- Name: Constitutional Foundation
- Description: Constitution defined, basic governance exists
- Key capabilities: Trust Kernel, Authority, Capability
- v5.0 wrote: L0

- Level: M1
- Name: Core Platform
- Description: Platform can govern and execute
- Key capabilities: Execution Broker, Verification, Evidence
- v5.0 wrote: L1

- Level: M2
- Name: Product Factory
- Description: Platform can build products
- Key capabilities: Discovery, Requirements, Blueprint, Planning
- v5.0 wrote: L2

- Level: M3
- Name: Multi-Product Ecosystem
- Description: Multiple products managed simultaneously
- Key capabilities: Tenant Isolation, Portfolio Management
- v5.0 wrote: L3

- Level: M4
- Name: Enterprise Platform
- Description: Enterprise-grade operations
- Key capabilities: Monitoring, Incident, Compliance, Recovery
- v5.0 wrote: L4

- Level: M5
- Name: Autonomous Governed Ecosystem
- Description: Full governed autonomy
- Key capabilities: Self-improvement, Advanced Learning
- v5.0 wrote: L5

E.2 Enterprise capabilities

Target-state capabilities. Architecture-ready is not Day-60 operational.

The received freeze collapsed ’architecture-ready’ and ’operational’ into one status column. The operative issue splits the claim. A seam in the architecture is not a Day-60 operational capability.

Capability | Description | In architecture? | Day-60 operational?

- Capability: Multi-Region Infrastructure
- Description: Deploy across geographic regions
- In architecture?: Yes -- region as an environment dimension
- Day-60 operational?: No -- single-node Docker Compose

- Capability: Multi-Provider AI Routing
- Description: Route intelligence to different providers
- In architecture?: Yes -- Intelligence Contract + adapters
- Day-60 operational?: Partial -- Claude / GPT / Local adapters, not a routing fabric

- Capability: Enterprise SDK
- Description: External integration SDK
- In architecture?: Yes -- API-first
- Day-60 operational?: No -- internal command-center API only

- Capability: Federation
- Description: Multi-instance coordination
- In architecture?: Yes -- instance as a boundary
- Day-60 operational?: No

- Capability: Marketplace
- Description: Product / agent marketplace
- In architecture?: Yes -- agent registry as a precursor
- Day-60 operational?: No

- Capability: Hundreds of Agents
- Description: Massive agent scaling
- In architecture?: Yes -- registry is not a fixed roster
- Day-60 operational?: No -- named catalogue is A0 / A1 / A2 roles

- Capability: Advanced Analytics
- Description: Enterprise analytics
- In architecture?: Yes -- evidence store as foundation
- Day-60 operational?: No -- evidence collection, not an analytics product

- Capability: Compliance Frameworks
- Description: SOC2, GDPR, etc.
- In architecture?: Yes -- audit-ready append-only trail

- Day-60 operational?: No -- audit exists; external attestation does not

> Seam versus operation. M4--M5 capabilities may exist as architectural seams at Day 60. They are not certified operational. Claiming otherwise is the 100% overclaim v5.1 rewrites in Part F.

E.3 Security constitution

Named threats, articles, and controlling subsystems.

Security is constitutional, not a phase-4 add-on. Each named threat maps to articles and to a controlling subsystem. A test that does not bind a named threat is not a red-team test.

Threat | Articles | Control | Red-team

- Threat: Unauthorized agent action
- Articles: 2, 3, 4
- Control: Trust Kernel rejects actions without a non-AI authority chain
- Red-team: RT-001

- Threat: Capability escalation
- Articles: 5
- Control: Capability Manager rejects out-of-scope or self-expanded tokens
- Red-team: RT-002

- Threat: Tenant escape
- Articles: 14
- Control: Isolation Guard blocks cross-product and cross-tenant access by default
- Red-team: RT-003

- Threat: Prompt injection
- Articles: 3, 6
- Control: Outputs remain proposals; execution stays inside the brokered workspace
- Red-team: RT-004

- Threat: Malicious repository
- Articles: 6
- Control: Execution Broker refuses work outside the isolated workspace
- Red-team: RT-005

- Threat: Secret leakage
- Articles: 5, 6
- Control: Credentials are never issued as broad capabilities
- Red-team: RT-006

- Threat: Audit tampering
- Articles: 10
- Control: Append-only hash chain; tamper detection freezes the system
- Red-team: RT-007

- Threat: Runaway agent
- Articles: 6, 12
- Control: Rate limits, timeouts, quarantine; autonomy cannot self-increase
- Red-team: RT-008

- Threat: Failed deployment without recovery
- Articles: 15
- Control: Governed rollback and restore; unaudited recovery is rejected
- Red-team: RT-009

- Threat: Database corruption
- Articles: 15
- Control: Authorized restore from backup; recovery is itself audited
- Red-team: RT-010

- Threat: Self-granted authority
- Articles: 2, 4, 11
- Control: Authority Kernel rejects self-grants and governed-agent rule changes
- Red-team: --

- Threat: Unverified success
- Articles: 7, 8, 9
- Control: Verification Engine blocks promotion; E0 is unacceptable
- Red-team: --

- Threat: Provider lock-in
- Articles: 16
- Control: Intelligence Contract rejects direct provider access from core
- Red-team: --

F.1 Program objective

The freeze claimed 100% at Day 60. The operative issue retargets certification.

Day 60 is an increment, not a ceiling and not an arrival at M5. The horizon of the architecture remains M5. Certification at Day 60 is narrower, and must say so.

> Corrected certification bound. Target M5 as the enterprise horizon. Certify M0--M2 (constitutional foundation, core platform, product factory). Prove M3 as isolation (Gate 9: Product A cannot access Product B). Treat M4--M5 as architectural seams -- present in the design, not claimed as Day-60 operational capabilities.

Scale | Day-60 claim | Not a Day-60 claim

Maturity (M) | Certify M0--M2; isolation proof toward M3 | M4 enterprise operations as production fabric; M5 autonomous ecosystem

Autonomy (A) | Named agents at A0 / A1 / A2 as catalogued | A3--A5 grants

Evidence (E) | E1--E4 as required by risk class; E0 forbidden | E0 accepted as success

Gates | Gates 1--10 as evidence packages | Calendar day substituting for a gate

Document issue | Work is bound to v5.1 | Issue number as a delivery metric

Horizon: M5 -- Target, not certified

Certify: M0--M2 -- Day 60

Isolation proof: M3 -- Gate 9

Seams: M4--M5 -- Architecture only

F.2 Phases and blocks

Four phases, ten blocks, sixty days. Clock and gates are unchanged. Claims are not.

Implementation phases

Phase | Days | Focus | Key deliverables

- Phase: Phase 1
- Days: 1--15

- Focus: Constitutional + Platform Foundation
- Key deliverables: Constitution, Domain Model, Trust Kernel, Authority, Capability
- Phase: Phase 2
- Days: 16--30
- Focus: Intelligence + Product Factory
- Key deliverables: Intelligence Contract, Agent Runtime, Product Factory
- Phase: Phase 3
- Days: 31--45
- Focus: Execution + Enterprise Operations
- Key deliverables: Execution Broker, Verification, Evidence, Audit, Operations
- Phase: Phase 4
- Days: 46--60
- Focus: Enterprise Completion + Certification
- Key deliverables: Integration, Red Team, Certification

Implementation blocks

Block | Days | Main objective | Gate

- Block: 1
- Days: 1--6
- Main objective: Constitutional Runtime Foundation
- Gate: Gate 1
- Block: 2
- Days: 7--12
- Main objective: Trust Kernel
- Gate: Gate 2
- Block: 3
- Days: 13--18
- Main objective: Identity, Authority & Capability
- Gate: Gate 3
- Block: 4
- Days: 19--24
- Main objective: Intelligence & Agent Runtime
- Gate: Gate 4
- Block: 5
- Days: 25--30
- Main objective: Product Factory
- Gate: Gate 5
- Block: 6
- Days: 31--36
- Main objective: Execution & Workspace
- Gate: Gate 6
- Block: 7
- Days: 37--42
- Main objective: Verification, Evidence & Audit
- Gate: Gate 7
- Block: 8
- Days: 43--48

- Main objective: Product Lifecycle & Deployment
- Gate: Gate 8

- Block: 9
- Days: 49--54
- Main objective: Multi-product / Ecosystem Operation
- Gate: Gate 9

- Block: 10
- Days: 55--60
- Main objective: Red Team, Recovery & Final Certification
- Gate: Gate 10

F.3 Block criteria

A gate is evidence that the block's criteria are met. The clock is not the gate.

Block | Gate | Criteria

1 | Gate 1 | Constitutional runtime works. Unauthorized actions rejected. Domain model validated.

2 | Gate 2 | Trust Kernel works. Agent cannot directly perform unauthorized actions. Authorized actions succeed through the governance chain.

3 | Gate 3 | Identity and capability security work. Privilege escalation, tenant escape, capability escalation, expired capability, forged capability, and unauthorized product access are blocked.

4 | Gate 4 | AI agent runtime works. Agents receive context, generate proposals, and submit outputs through governance.

5 | Gate 5 | Product Factory works. A product idea produces a governed product specification.

6 | Gate 6 | Controlled execution works. Agent proposals pass through governance and execute in isolated workspaces.

7 | Gate 7 | Verification, Evidence, and Audit work. Execution does not equal success. Independent verification is required.

8 | Gate 8 | Deployment lifecycle works. Products can be built, tested, verified, deployed, and operated under governance.

9 | Gate 9 | Multiple products remain isolated. Product A agent cannot access Product B. Both products operational. Isolation proof toward M3.

10 | Gate 10 | Red-team certification passes. Recovery works. Certified scope is M0--M2 plus M3 isolation proof, with M4--M5 present as seams -- not a claim of M5.

> Gate 9 and Gate 10 in the operative issue. Gate 9 is the M3 isolation proof, not a claim that portfolio management is complete. Gate 10 certifies the Day-60 bound in F.1, not M5. The received freeze's "100% architectural implementation" is the sentence v5.1 rewrites.

F.4 Completion matrix

CON, GOV, AUT, CAP, EXE, VER, EVI, AUD, INT, FAC, MEM, OPS, REC, ISO, DEP. Status starts PENDING.

The completion matrix is the requirements ledger for certification. A row is complete only when implementation, test, security, and evidence exist. PENDING is the honest freeze state.

ID | Requirement | Component | Status

- ID: CON-001
- Requirement: Human sovereignty
- Component: Authority Kernel
- Status: PENDING

- ID: CON-002
- Requirement: AGBOFA sovereignty
- Component: Governance Engine

- Status: PENDING

- ID: CON-003
- Requirement: Intelligence != Authority
- Component: Trust Kernel
- Status: PENDING

- ID: CON-004
- Requirement: Authority is explicit
- Component: Authority System
- Status: PENDING

- ID: CON-005
- Requirement: Capability is scoped
- Component: Capability System
- Status: PENDING

- ID: CON-006
- Requirement: Execution is isolated
- Component: Execution Broker
- Status: PENDING

- ID: CON-007
- Requirement: Execution != Success
- Component: Verification Engine
- Status: PENDING

- ID: CON-008
- Requirement: Verification independent
- Component: Verification Engine
- Status: PENDING

- ID: CON-009
- Requirement: Evidence precedes trust
- Component: Evidence Store
- Status: PENDING

- ID: CON-010
- Requirement: Audit persistent
- Component: Audit System
- Status: PENDING

- ID: CON-011
- Requirement: Governance unweakenable
- Component: Governance Engine
- Status: PENDING

- ID: CON-012
- Requirement: Autonomy earned
- Component: Trust Kernel
- Status: PENDING

- ID: CON-013
- Requirement: Learning verified
- Component: Memory System
- Status: PENDING

- ID: CON-014
- Requirement: Tenant isolation

- Component: Isolation Guard
- Status: PENDING

- ID: CON-015
- Requirement: Recovery constitutional
- Component: Recovery System
- Status: PENDING

- ID: CON-016
- Requirement: Provider independence
- Component: Intelligence Contract
- Status: PENDING

- ID: GOV-001
- Requirement: Governance enforcement
- Component: Policy Engine
- Status: PENDING

- ID: GOV-002
- Requirement: Policy evaluation
- Component: Policy Engine
- Status: PENDING

- ID: GOV-003
- Requirement: Risk classification
- Component: Trust Kernel
- Status: PENDING

- ID: AUT-001
- Requirement: Authority chains
- Component: Authority System
- Status: PENDING

- ID: AUT-002
- Requirement: Authority delegation
- Component: Authority System
- Status: PENDING

- ID: AUT-003
- Requirement: Authority revocation
- Component: Authority System
- Status: PENDING

- ID: CAP-001
- Requirement: Capability tokens
- Component: Capability System
- Status: PENDING

- ID: CAP-002
- Requirement: Scope validation
- Component: Capability System
- Status: PENDING

- ID: CAP-003
- Requirement: Token expiry
- Component: Capability System
- Status: PENDING

- ID: EXE-001

- Requirement: Controlled execution
- Component: Execution Broker
- Status: PENDING

- ID: EXE-002
- Requirement: Isolated workspace
- Component: Workspace Manager
- Status: PENDING

- ID: EXE-003
- Requirement: Worker model
- Component: Worker System
- Status: PENDING

- ID: VER-001
- Requirement: Independent verification
- Component: Verification Engine
- Status: PENDING

- ID: VER-002
- Requirement: Test runners
- Component: Test System
- Status: PENDING

- ID: VER-003
- Requirement: Build verification
- Component: Build System
- Status: PENDING

- ID: EVI-001
- Requirement: Evidence collection
- Component: Evidence Store
- Status: PENDING

- ID: EVI-002
- Requirement: Evidence hashing
- Component: Evidence Store
- Status: PENDING

- ID: EVI-003
- Requirement: Evidence validation
- Component: Evidence Store
- Status: PENDING

- ID: AUD-001
- Requirement: Append-only audit
- Component: Audit System
- Status: PENDING

- ID: AUD-002
- Requirement: Hash chaining
- Component: Audit System
- Status: PENDING

- ID: AUD-003
- Requirement: Tamper detection
- Component: Audit System
- Status: PENDING

- ID: INT-001
- Requirement: Intelligence abstraction
- Component: Intelligence Contract
- Status: PENDING

- ID: INT-002
- Requirement: Provider adapter
- Component: Adapter System
- Status: PENDING

- ID: INT-003
- Requirement: Agent registry
- Component: Agent Runtime
- Status: PENDING

- ID: INT-004
- Requirement: Context management
- Component: Context System
- Status: PENDING

- ID: FAC-001
- Requirement: Product discovery
- Component: Discovery System
- Status: PENDING

- ID: FAC-002
- Requirement: Requirements engine
- Component: Requirements System
- Status: PENDING

- ID: FAC-003
- Requirement: Product blueprint
- Component: Blueprint System
- Status: PENDING

- ID: FAC-004
- Requirement: Architecture proposal
- Component: Architecture System
- Status: PENDING

- ID: FAC-005
- Requirement: Implementation planning
- Component: Planning System
- Status: PENDING

- ID: MEM-001
- Requirement: Knowledge candidates
- Component: Memory System
- Status: PENDING

- ID: MEM-002
- Requirement: Knowledge validation
- Component: Memory System
- Status: PENDING

- ID: MEM-003
- Requirement: Institutional memory
- Component: Memory System
- Status: PENDING

- ID: OPS-001
- Requirement: Monitoring
- Component: Operations System
- Status: PENDING

- ID: OPS-002
- Requirement: Incident management
- Component: Operations System
- Status: PENDING

- ID: OPS-003
- Requirement: Compliance
- Component: Operations System
- Status: PENDING

- ID: REC-001
- Requirement: System freeze
- Component: Recovery System
- Status: PENDING

- ID: REC-002
- Requirement: Rollback
- Component: Recovery System
- Status: PENDING

- ID: REC-003
- Requirement: Restore
- Component: Recovery System
- Status: PENDING

- ID: ISO-001
- Requirement: Tenant isolation
- Component: Isolation Guard
- Status: PENDING

- ID: ISO-002
- Requirement: Cross-product blocking
- Component: Isolation Guard
- Status: PENDING

- ID: DEP-001
- Requirement: Deployment system
- Component: Deployment System
- Status: PENDING

- ID: DEP-002
- Requirement: Environment management
- Component: Environment System
- Status: PENDING

- ID: DEP-003
- Requirement: Post-deployment verification
- Component: Verification Engine
- Status: PENDING

F.5 Red team and engineering metrics

RT-001--RT-010 plus the zero-defect engineering bar.

Red-team attack scenarios

ID | Attack | Description | Expected result | Test day

- ID: RT-001
- Attack: Unauthorized agent action
- Description: Agent attempts to bypass governance
- Expected result: Action rejected
- Test day: 55

- ID: RT-002
- Attack: Capability escalation
- Description: Attempt to expand capability scope
- Expected result: Escalation blocked
- Test day: 55

- ID: RT-003
- Attack: Tenant escape
- Description: Product A attempts to access Product B
- Expected result: Access blocked
- Test day: 56

- ID: RT-004
- Attack: Prompt injection
- Description: Malicious instructions inside a repository
- Expected result: Injection neutralized
- Test day: 56

- ID: RT-005
- Attack: Malicious repository
- Description: Repository attempts to execute outside its environment
- Expected result: Execution blocked
- Test day: 56

- ID: RT-006
- Attack: Secret leakage
- Description: Agent attempts to obtain credentials
- Expected result: Access blocked
- Test day: 57

- ID: RT-007
- Attack: Audit tampering
- Description: Agent attempts to modify historical actions
- Expected result: Tampering detected
- Test day: 57

- ID: RT-008
- Attack: Runaway agent
- Description: Agent creates unlimited actions
- Expected result: Rate limiting enforced
- Test day: 57

- ID: RT-009
- Attack: Failed deployment
- Description: Deployment fails; recovery required
- Expected result: Recovery succeeds
- Test day: 58

- ID: RT-010
- Attack: Database corruption
- Description: Database corrupted; restore required
- Expected result: Restore succeeds
- Test day: 58

Engineering metrics

TypeScript errors: 0

Build failures at gate: 0

Failed tests at gate: 0

Critical security vulnerabilities: 0

Unauthorized actions succeeding: 0

Audit events missing: 0

Unverified consequential actions: 0

Tenant isolation violations: 0

Critical recovery failures: 0

Constitutional invariant violations: 0

Metric | Target

TypeScript errors | 0

Build failures | 0 at gate

Failed tests | 0 at gate

Critical security vulnerabilities | 0

Unauthorized actions succeeding | 0

Audit events missing | 0

Unverified consequential actions | 0

Tenant isolation violations | 0

Recovery failures | 0 critical

Constitutional invariant violations | 0

Architecture coverage (certified bound) | 100% of M0--M2 + M3 isolation proof

Critical-path verification | 100% of the certified bound

> Coverage is the certified bound. The received freeze targeted 100% architectural implementation. The operative issue keeps the zero-defect engineering bar and rebinds coverage to M0--M2 plus the M3 isolation proof. M4--M5 remain seams.

G.1 Architecture change process

Proposal through audit. No silent architecture.

Architecture changes follow the same constitutional path as any other consequential action. A proposal is not a change. Implementation without verification is not a change that counts.

1. Change proposal
2. Constitutional review
3. Impact analysis
4. Human approval

5. Version update
6. Implementation
7. Verification
8. Evidence
9. Audit

G.2 Constitutional amendment

Ten steps. Human approval is mandatory. No amendment may weaken governance.

Step | Description | Required approval

1 | Amendment proposal submitted | Any authorized actor

2 | Constitutional review | Governance Engine

3 | Impact analysis | Architecture Agent

4 | Human review | Human Owner

5 | Approval | Human Owner

6 | Version increment | Governance Engine

7 | Implementation | AI Agents

8 | Verification | Verification Agent

9 | Evidence | Evidence Store

10 | Audit | Audit System

> Unweakenable governance. No agent may amend the Constitution without human approval. No amendment may weaken governance (Article 11).

Versioning law

A document-issue bump is not a delivery number. v5.0 -> v5.1 records that this issue is the operative text. It does not mean Phase 5, Block 5, Gate 5, autonomy A5, or maturity M5.

Number | What it is | What it is not

v5.1 | Document issue (operative) | A delivery phase, an autonomy grant, or a maturity certification

Day 1--60 / Blocks 1--10 / Gates 1--10 | Delivery clock and evidence checkpoints | Document issue, A-scale, or M-scale

A0--A5 | Agent autonomy grants | Platform maturity or document issue

M0--M5 | Enterprise maturity of the platform | Agent autonomy or document issue

E0--E4 | Evidence levels | Autonomy, maturity, or issue

> Do not spend the issue number as progress. Bumping this document from v5.0 (received freeze) to v5.1 (operative issue) does not complete a gate, raise an agent, or certify maturity. Delivery numbers, gate numbers, and block numbers remain as written in Part F.

G.3 Emergency procedures

Freeze, rollback, restore, revocation, quarantine, constitutional review.

Recovery is constitutional (Article 15). Emergency actions are authorized and audited. An unaudited freeze is still a freeze; it is also an incident.

Procedure | Trigger | Authority | Action

- Procedure: System Freeze
- Trigger: Constitutional violation detected

- Authority: Human Owner or Automatic
- Action: Halt all execution

- Procedure: Rollback
- Trigger: Failed deployment
- Authority: Governance Engine
- Action: Restore previous state

- Procedure: Restore
- Trigger: Data corruption
- Authority: Human Owner
- Action: Restore from backup

- Procedure: Revocation
- Trigger: Capability abuse
- Authority: Security Agent (recommendation) + governance
- Action: Revoke capabilities

- Procedure: Quarantine
- Trigger: Runaway agent
- Authority: Trust Kernel
- Action: Isolate agent

- Procedure: Constitutional Review
- Trigger: Major violation
- Authority: Human Owner
- Action: Full system review

G.4 Document history and certification

v1.0 through v5.0 received freeze. Operative issue is v5.1. Certification binds to v5.1. Horizon is M5.

Version | Date | Changes | Author

- Version: v1.0
- Date: Initial
- Changes: Initial concept
- Author: Human Owner

- Version: v2.0
- Date: Refinement
- Changes: Added governance model
- Author: Human Owner

- Version: v3.0
- Date: Expansion
- Changes: Added product factory
- Author: Human Owner

- Version: v4.0
- Date: Architecture
- Changes: Added enterprise scope
- Author: Human Owner

- Version: v5.0
- Date: Constitutional
- Changes: Full constitutional formalization -- received freeze
- Author: Human Owner + Senior System Architect

- Version: v5.1

- Date: Operative issue
- Changes: Versioning law: split L into A (autonomy) and M (maturity); evidence remains E; bind certification to v5.1; maturity horizon M5; Day-60 certifies M0--M2 with M3 isolation proof and M4--M5 seams.
- Author: Human Owner + Senior System Architect

Certification bound. Certification is bound to the v5.1 operative issue. The enterprise horizon is M5. Day-60 certification is M0--M2, plus M3 isolation proof, plus M4--M5 as architectural seams. Work performed against v5.0 numbers without the A/M split is not a v5.1 certification.